Backend Assignment - Filesystem Management (Go)

General Notes

- Step 3 (CLI Tool) is optional
- Each step should be implemented in a separate pull request (PR).
- Follow Golang best practices and adhere to Git conventions throughout the assignment.

General Description

You are tasked with building a file storage service using Golang, focusing on file upload, retrieval, and metadata management. The service should be implemented using a microservices architecture. A queue system will be integrated to handle real-time updates. The assignment is divided into three steps, with the final step being optional.

Golang Requirement

- All code for the microservices must be written in Golang.
- The CLI tool (if implemented) can be written in any language of your choice.

Submission Guidelines

1. Create a public or private repository on GitHub.
2. Push your code to the repository, ensuring that the project is well-organized with a clear directory structure.
3. Include a `README.md` file with instructions on how to run the services using Docker Compose and any other relevant details.
4. Share the repository with `mayk0gan`.
5. Open a pull request in the repository for review.

Assignment Steps

Step 1: Core Microservices and API Implementation

Objective

Build the foundation of a file storage service with basic microservices architecture, focusing on file upload, retrieval, and deletion.

Requirements

1. **Microservices Architecture**
    - **Storage Service**
        - **Responsibilities:** Handle operations related to file storage, including uploading, retrieving, and deleting files.
        - **File Upload:** Save files to MinIO with a unique identifier (UUID).
        - **Concurrency:** Ensure that the service can handle multiple file uploads simultaneously using Goroutines and channels.
        - **Endpoints**
            - `POST /upload`: Upload a file to MinIO.
            - `GET /file/{id}`: Retrieve a file from MinIO using the file's UUID.
            - `DELETE /file/{id}`: Delete a file from MinIO using the file's UUID.
    - **Metadata Service**
        - **Responsibilities:** Manage metadata associated with the files, such as file name, size, upload timestamp, and storage location.
        - **Database:** Store metadata in a PostgreSQL database.
        - **Endpoints**
            - `POST /metadata`: Save file metadata after the file is uploaded.
            - `GET /metadata/{id}`: Retrieve metadata for a specific file using its UUID.
            - `GET /files/`: List all files with their metadata (file location, size, upload time, etc.).

Deliverables:

- A basic implementation of the Storage and Metadata services with the described APIs.

- Unit tests for key functionalities.
- A `README.md` file with instructions on how to run the services using Docker Compose, including setting up PostgreSQL.

Important: All code should be written in Golang.

Step 2: Integration of a Queue System for Real-Time Updates

Objective:
Enhance the file storage service by integrating a queue system to handle real-time updates.

Requirements:

1. **Queue System for Real-Time Updates:**
    - Implement a queue system to broadcast real-time updates to clients when certain actions occur (e.g., a file is uploaded, deleted, or updated).
    - Clients can subscribe to these updates and receive notifications, allowing the system to provide a more interactive experience.
    - You can choose any queue system that supports local development or Docker-based deployment. Examples include:
        - **Redis Pub/Sub**
        - **RabbitMQ**
        - **Kafka**

Deliverables:

- Enhanced services with a queue system for real-time updates.
- Integration tests to ensure the services interact correctly with the chosen queue system.
- Updates to the `README.md` file explaining the new architecture and how to test these enhancements.

Important: All code should be written in Golang.

Step 3: CLI Tool (Optional)

Objective:
Develop a command-line interface (CLI) tool that interacts with the file storage service, providing a user-friendly way to upload, retrieve, delete, and list files.

Requirements:

1. **CLI Tool:**
    - Implement a CLI that allows users to:
        - **Upload a file:** Command for uploading a file to the Storage Service.
        - **Retrieve a file:** Command to fetch and download a file using its UUID.
        - **Delete a file:** Command to delete a file by its UUID.
        - **List files:** Command to list all files with their metadata.

Deliverables:

- A CLI tool with commands for interacting with the file storage service.
- Documentation in the `README.md` explaining how to use the CLI tool.

Important: The CLI tool can be written in any language of your choice.